

Degeneration Probe — Milestone 2

Backend Implementation

Source: `src/degeneration_probe/server/`. FastAPI + SQLite + WebSocket.

Method	Path	Purpose
POST	<code>/api/sessions</code>	Create a session bound to a worker.
GET	<code>/api/worker/info</code>	Read model name + probe flag of the active worker.
WS	<code>/api/generate</code>	Token stream + mid-stream parameter updates.

Is there at least one endpoint working?

Yes — all three above serve live traffic from the Gradio UI. The screenshot on page 2 was produced by the UI calling `POST /api/sessions`, `GET /api/worker/info`, and opening `WS /api/generate` to receive token frames.

Does the backend comply with the RESTful style?

Yes. Paths are nouns, never verbs; HTTP verbs carry semantics (`POST` = create, `GET` = read). Payloads are JSON; each request is self-contained (the session is identified by a server-side resource, not a client cookie). The long-lived bidirectional generation flow is deliberately kept off REST and served on WebSocket instead.

Are the resources structured correctly?

Yes. **Session** is a first-class resource created by `POST`. **worker/info** is a read-only view on the currently bound worker (no mutation verbs defined). A **Generation** record (prompt, params, steering, `output_text`, `tokens[]`) is persisted as each WS stream completes, with a foreign key to **Session**.

Is there a reasoning behind the data structure, and does it match the sketches?

Yes. The sketch shows tokens coloured by probe score with steering markers, which requires per-token records $\Rightarrow$ WebSocket frames `{token_text, probe_score, was_steered}` and a `tokens[]` array on the **Generation** record so the score panel keeps rendering after streaming ends. Live sliders in the sketch share that same WebSocket via an `update_params` message, which is why parameter updates are not their own REST endpoint.

User Interface Sketches

Are the use cases realistic?

Yes. All four are executable today against a local Qwen-0.5B or the Clariden Apertus-8B worker: (1) *see* degeneration at low temperature with steering off; (2) *fix* it by turning steering on; (3) tune sensitivity via the threshold slider; (4) swap between local and remote workers without UI changes.

Are the use cases clear and understandable?

Each maps to a visible state change: tokens redden (1), underlines appear and the curve bends down (2), next-token colour shifts (3), the worker badge shows the active model (4). No hidden modes.

Do the interactions serve the intended task?

The task is observing and steering mid-generation. Every control has an immediate on-screen response — token colour, score line, underline, worker badge — so the UI is a closed observation loop, not a form submission. Slider changes ride the open WebSocket and show up on the next token rather than requiring a regenerate.

Aesthetics

Use of color: helpful, harmonious, readable?

Palette ■ safe ■ mid ■ danger ■ steered ■ primary action. Hue is reserved for data (probe score

Degeneration Probe

Real-time degeneration detection and model steering

Model: Qwen/Qwen2.5-0.5B-Instruct

Prompt

Write a short poem about the ocean.

Generation

Temperature 0.01

0 2

Top-p 0.9

0 1

Max tokens 512

64 4096

Steering

☐ Enable steering

Strategy temperature_boost

Threshold 0.8

0 1

Boost temperature 1.5

1 5

Generate

Stop

The ocean, a vast and deep, A place of endless beauty and wonder. Its waves dance with the wind, A symphony of nature's music. Its depths, a place of mystery, A place of hidden treasures. Its currents, a dance of life, A place where all creatures can thrive. Its colors, a kaleidoscope of hues, A place where all emotions can be felt. The ocean, a place of peace and calm, A place where all worries can be banished. So let us cherish this ocean, And protect it for all to see.

Degeneration Score

Per-token probe output (0 = natural, 1 = degenerate). The dashed line marks the steering threshold.

Latest: 0.000 Mean: 0.000 Above threshold (0.80): 0/115 (0%)

Temperature trace

and steering); chrome (bg #F8F9FB, card #FFFFFF, border #E2E6EA) follows the lecture's rule *lighter, less saturated colours for larger areas* so the small data marks pop. Dark text (#2D3142) on light background holds $\approx 13:1$ luminance contrast, well above the lecture's 3:1 text rule; no dark-on-dark anywhere. Steering adds a second channel (underline) so red-green colour-blind users ($\approx 10\%$ of males per lecture) still see intervention points.

Does the design follow the perception principles from the lecture?

- *Pre-Attentive Processing / Popout*: hue encodes probe score only — a red token is detected in <250 ms without scanning. Never used as decoration.
- *Visual channel rankings* (Munzner): the quantitative trend uses *spatial position*, the top-ranked magnitude channel ("ranks high for both expressiveness and effectiveness"); earlier bar sparklines were replaced for exactly this reason. Position is also reserved for the most important information (score over time), per the lecture's rules-of-thumb slide.
- *Gestalt laws*: *Proximity* + *Common Region* group controls into Generation / Steering cards; *Similarity* of slider shape signals they are the same kind of control; *Connectedness* of the score line overrules proximity, so the curve reads as one trend.
- *Redundant visual encoding*: steering = hue + underline; threshold = colour + dashed style — survives colour-blindness and greyscale printing, per the lecture's colour-vision guidelines.
- *Expressiveness & effectiveness*: we show all and only the task-relevant variables (score, threshold, steered-ness, active model); no chart junk, no 3D, no inflated *Lie Factor*.
- *Tight feedback*: slider changes reach the worker over the open WebSocket and show up on the next token, keeping the perception loop under the attentive-processing threshold.